UNIVERSIDAD POPULAR DEL CESAR

Ingeniería de Sistemas e Informática

GUÍA DEL TALLER

Refactorización a Código Limpio

Sistema de Facturación — Restaurante El Buen Sabor

Programación de Computadores III · SS462 · Cuarto Semestre

Docente: Ing. Alfredo David Bautista Romero

1. ¿Qué van a encontrar?

El proyecto RestauranteBuenSabor es un sistema de facturación para un restaurante que funciona correctamente, pero está escrito con el máximo de malas prácticas posibles. Su misión es refactorizarlo hasta convertirlo en código limpio, sin alterar el comportamiento del programa.

> ⚠ *El programa produce resultados correctos. El problema no es que falle — el problema es que es imposible de mantener, entender y modificar. Eso es exactamente lo que pasa en la industria real.*

1.1 Estructura de archivos entregada

```
RestauranteBuenSabor/
├── src/
│   └── restaurante/
│       ├── Main.java        — punto de entrada y menú interactivo
│       ├── Datos.java       — almacén global de datos del sistema
│       ├── Proceso.java     — lógica de cálculo de la factura
│       ├── Utilidades.java  — métodos auxiliares
│       └── Imprimir.java    — todo lo que se imprime en pantalla
└── README.md                — instrucciones para compilar y ejecutar
```

2. ¿Qué hace el sistema?

El programa simula la operación de caja de un restaurante. Al ejecutarlo aparece un menú con 6 opciones que se repite en bucle hasta que el usuario elige salir:

```
========================================
    RESTAURANTE EL BUEN SABOR
    Calle 15 #8-32, Valledupar
    NIT: 900.123.456-7
========================================
1. Ver carta
2. Agregar producto al pedido
3. Ver pedido actual
4. Generar factura
5. Nueva mesa
0. Salir
========================================
Seleccione una opcion:
```

2.1 Carta del restaurante (Opción 1)

#	Producto	Precio
1	Bandeja Paisa	$32.000
2	Sancocho de Gallina	$28.000
3	Arepa con Huevo	$8.000

4	Jugo Natural	$7.000
5	Gaseosa	$4.500
6	Cerveza Poker	$6.000
7	Agua Panela	$3.500
8	Arroz con Pollo	$25.000

2.2 Reglas de negocio (Opción 4 — Generar factura)

- Subtotal: suma de precio × cantidad para cada producto pedido.
- Descuento 5%: se aplica si hay más de 3 productos diferentes en el pedido.
- IVA 19%: se calcula sobre el subtotal después del descuento.
- Propina 10%: se aplica si el subtotal (antes del IVA) supera $50.000.
- Total: subtotal con descuento + IVA + propina.

Ejemplo de factura generada (Bandeja Paisa x2 + Jugo Natural x2):

```
FACTURA No. 001
----------------------------------------
Bandeja Paisa        x2      $64.000
Jugo Natural         x2      $14.000
----------------------------------------
Subtotal:                   $78.000
IVA (19%):                  $14.820
Propina (10%):              $ 9.282
----------------------------------------
TOTAL:                      $102.102
```

3. Catálogo de Malas Prácticas

A continuación, se documenta cada mala práctica presente en cada archivo, con su ubicación y descripción. Usen esto como lista de verificación durante la refactorización.

3.1 Datos.java — El God Object

Almacena todos los datos del sistema en variables estáticas públicas. Cualquier clase puede leer y modificar cualquier dato directamente, sin control ni encapsulamiento.

#	Problema	Dónde	Descripción
1	Arrays paralelos	nom[], p[], cant[] L. 12-14	Tres arrays separados representan el mismo concepto (un producto). Si se reordena uno, los otros quedan desfasados. Deben reemplazarse por una clase Producto.
2	Todo public static	Todas las variables L. 12-18	Cualquier clase puede leerlas y sobrescribirlas sin restricción. Viola encapsulamiento. Deben ser private con getters/setters.
3	Nombre hardcodeado	String x = "El Buen Sabor" L. 17	El nombre del restaurante en una variable llamada x — nombre críptico y valor duplicado en otros 4 lugares.
4	Variable tmp sin propósito	String tmp = "" L. 18	Se declara como campo de clase pero nunca se usa para ningún cálculo real en Datos. Es dead code disfrazado de campo.

5	Sin constructor ni métodos	Clase entera L. 11-19	Contenedor plano de variables globales. Antipatrón Data Clump: no tiene comportamiento ni encapsulamiento alguno.
6	Nombres de 1-3 letras	nom, p, cant, ms, est, tot, nf, x, tmp L. 12-18	Nueve campos crípticos. El lector debe adivinar que p=precio, ms=mesa, est=estado, nf=número de factura.

3.2 Proceso.java — El método que hace todo

Contiene la lógica de cálculo del total de la factura en un único método de más de 80 líneas.

#	Problema	Dónde	Descripción
7	hacerTodo() con múltiples responsabilidades	Método hacerTodo() L. 12-46	Acumula subtotal, cuenta productos, aplica descuento, calcula IVA, aplica propina y actualiza estado global. Viola SRP. Debe dividirse en al menos 4 métodos.
8	IVA duplicado 3 veces	iva=aux*0.19 (3 ramas distintas) L. 24, 30, 38 y 43	La fórmula aparece idéntica en las tres ramas condicionales. Cambiar la tasa exige modificar 3 lugares. Debe ser calcularIva(double base).
9	Magic numbers sin nombre	0.05, 0.19, 0.10, 50000, 3, 0.01 L. 22, 24, 27, 30, 38, 41, 43, 62	Seis literales sin contexto. 0.01 en procesar() es especialmente misterioso — no corresponde a ninguna regla de negocio documentada.
10	Condicionales anidados 4 niveles	if(cont>3){ if(sub>0){ if(aux>50000){... L. 20-44	Cuatro niveles hacen imposible razonar sobre todas las combinaciones. Se aplana con cláusulas de guarda y métodos auxiliares.
11	Código comentado 'versión anterior'	// version anterior - no borrar L. 32-35	4 líneas de implementación anterior con la advertencia "no borrar". Los umbrales difieren (40000 vs 50000), lo que genera confusión adicional.
12	procesar() con 7 parámetros	procesar(double a, b, c, d, e, int f, boolean g) L. 47	Imposible saber al llamarlo qué representa cada argumento. Clean Code recomienda máximo 3. Debe usarse un objeto de parámetros.
13	Variable aux innecesaria	aux=sub-(sub*0.05) / tmp=iva en procesar() L. 48, 55-56	Intermediarios que no añaden claridad. return res es equivalente a aux2=res; return aux2.
14	Comentarios que repiten el código	// multiplica precio por cantidad antes de sub=sub+Datos.p[i]*Datos.cant[i] L. 16, 25, 29, 49, 52, 54, 58	El comentario dice exactamente lo que el código ya dice. Con nombres expresivos los comentarios sobran.
15	Comentarios de cierre inútiles	}// fin while, }// fin if sub>0, }// fin if-else cont L. 19, 31, 44	Síntoma de métodos demasiado largos. La solución es extraer métodos, no etiquetar llaves.

3.3 Utilidades.java — Nombres engañosos

Contiene métodos auxiliares: calcular, validar y reiniciar el pedido.

#	Problema	Dónde	Descripción
16	calcular() con 7 parámetros	calcular(double pr, cn, dc, iv, pp, int ni, boolean ap) L. 12	El parámetro ni (número de ítems) nunca se usa en el cuerpo del método. Es un parámetro fantasma que ocupa espacio sin cumplir función.

17	validar() con efecto secundario oculto	if(cont==0){ Datos.tot=0; Datos.tmp=""; } L. 23-30	El método se llama validar() pero además resetea datos globales. Comportamiento sorpresa que viola el Principio de Menor Asombro (POLA).
18	Nombre del restaurante hardcodeado (3ra vez)	System.out.println("RESTAURANTE EL BUEN SABOR - calculo aplicado") L. 20	Dentro de un método de cálculo se imprime presentación. Doble violación: DRY y SRP.
19	Bloque de ~20 líneas de código comentado	Inicio de reiniciar() L. 32-47	Un método calcOld() completo y un bloque alternativo de cálculo comentados ocupan más espacio que el código funcional real del método (5 líneas activas).
20	Variables tmp y aux2 redundantes	tmp=res*iv; res=res+tmp; / aux2=res; return aux2; L. 13, 21-22	Ambas pueden eliminarse: res=res+(res*iv) y return res son equivalentes directos.

3.4 Imprimir.java — Código copiado y pegado

Contiene todos los métodos de impresión en consola.

#	Problema	Dónde	Descripción
21	imprimirFacturaCompleta() con 3 responsabilidades	Método completo L. 34-78	(1) Recalcula subtotal/IVA/propina, (2) imprime la factura, (3) actualiza estado: Datos.nf++, Datos.est=0, Datos.tot=tot. Un método de impresión no debería tener efectos secundarios sobre el sistema.
22	Cálculo del total duplicado	imprimirFacturaCompleta() e imprimirFacturaResumen() L. 35-50 y L. 80-95	~15 líneas de cálculo copiadas literalmente. Cambiar una regla (ej. IVA al 21%) exige actualizar ambos métodos — fuente garantizada de bugs.
23	80% del código idéntico entre métodos	Ambos métodos de factura L. 34-78 vs L. 79-110	Mismo cálculo, mismo encabezado, mismos printf. La única diferencia es que Completa imprime los ítems y Resumen no. Debe haber un único método con parámetro.
24	Nombre del restaurante hardcodeado (4ta vez)	"RESTAURANTE EL BUEN SABOR" aparece 2 veces en este archivo L. 14, 53-55, 98-100	El literal del nombre del negocio aparece en ambos métodos de factura, sumando ya 4 ocurrencias en el proyecto.
25	Formato printf repetido sin constante	"%-27s $%,.0f%n" × 5 veces L. 19, 28, 33, 62, 65-70, 104-109	Si el ancho de columna cambia de 27 a 30, hay que modificar cinco líneas. Debe ser static final String FMT_MONTO = "%-27s $%,.0f%n";

3.5 Main.java — El caos central

Punto de entrada del programa. El método main mezcla presentación, validación y lógica de negocio en más de 100 líneas.

#	Problema	Dónde	Descripción
26	Método main de 100+ líneas	Método main() completo L. 14-132	Gestiona: encabezado, bucle de menú, lectura de opción, validación, lógica de mesa, procesamiento, manejo de errores y limpieza de estado. Cada opción debería ser un método separado.
27	Dos Scanner para el mismo System.in	Scanner sc2=new Scanner(System.in)	El propio código lo admite con // limpiar buffer con segundo scanner - innecesario. Crea el Scanner y ni lo cierra.

		dentro del bloque else L. 15 y L. 123	
28	Variables sin uso	aux, tmp, m, continuar L. 16	Las cuatro se asignan pero nunca influyen en ninguna decisión: continuar siempre es true y nunca se lee, m guarda un precio que no se imprime ni usa.
29	Lógica de negocio en el main	Datos.cant[n-1]=Datos.cant[n-1]+c; / if(Datos.est==0)... L. 51-57, 63, 96-98	El main modifica directamente arrays y estados de datos. La lógica "si la mesa no está activa, pedir número de mesa" pertenece a una clase de servicio.
30	Nombre del restaurante hardcodeado (5ta vez)	Encabezado inicial del main L. 18-20	Quinta y última ocurrencia del literal. Debe existir una única constante NOMBRE_RESTAURANTE referenciada desde todos los puntos.

3.6 Problemas de formato visual (todos los archivos)

#	Problema	Dónde	Descripción
31	Sin indentación	Todos los .java	Cuerpo de clase, métodos y bloques al mismo nivel visual. Imposible identificar la estructura a primera vista.
32	Llaves inconsistentes	Todos los .java	A veces llave al final de línea, a veces en línea nueva, sin un patrón definido.
33	Llaves de cierre apiladas	Todos los .java	Múltiples llaves de cierre aparecen en la misma línea (ej: i++;}// fin while o cont=cont+1;} i++;}) mezcladas con comentarios. Esto oculta la estructura de bloques.
34	Múltiples statements por línea	Todos los .java	Instrucciones como public static int ms=0;public static int est=0; o double sub=0;double iva=0;double tot=0; acumulan varias declaraciones en una sola línea, dificultando la lectura y el debugging.

Total: 34 malas prácticas identificadas y documentadas en el proyecto.

Referencia principal: Robert C. Martin — Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall, 2009.

Fuente complementaria: Shahan Chowdhury — The Clean Code Handbook. FreeCodeCamp, Enero 2025.